

PRACTICAL : WEEK 10

Task -1:

Investigate inode structures using commands such as ls -i, stat, and find. Create hard links and symbolic links and explain their impact on inode allocation.

CODE:

Program/Command Sequence: Investigation of Inodes, Hard Links and Symbolic Links

```
# Create a test file
echo "This is an inode demonstration file." > original.txt

# Display inode number
ls -i original.txt

# Display detailed inode information
stat original.txt

# Find file using inode number
find . -inum <inode_number>

# Create a hard link
ln original.txt hardlink.txt

# Compare inode numbers
ls -li original.txt hardlink.txt

# Modify the hard link
echo "Data added through hard link." >> hardlink.txt

# Verify that original file is also modified
cat original.txt

# Create a symbolic link
ln -s original.txt symlink.txt

# Display inode information of all files
ls -li original.txt hardlink.txt symlink.txt

# Display symbolic link information
stat symlink.txt
```

Follow symbolic link and display target information

stat -L symlink.txt

Find all files sharing the original inode

find . -inum <original_inode_number>

Display final inode and link information

stat original.txt

OUTPUT:

```
ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week10
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ mkdir -p /mnt/d/OSSP_Programs/Practical/Week10
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ echo "This is an inode demonstration file." > original.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ cat original.txt
This is an inode demonstration file.
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ ls -i original.txt
2251799813686402 original.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ stat original.txt
  File: original.txt
  size: 37          Blocks: 0          IO Block: 512    regular file
Device: 0,71      Inode: 2251799813686402 Links: 1
Access: (0777/-rwxrwxrwx)  Uid: ( 1000/  ananya)   Gid: ( 1000/  ananya)
Access: 2026-09-26 03:46:32.657842400 +0000
Modify: 2026-09-26 03:46:32.657842400 +0000
Change: 2026-09-26 03:46:32.657842400 +0000
 Birth: -
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ ls -i original.txt
2251799813686402 original.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ find . -inum 123456
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ find . -inum 2251799813686402
./original.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ ln original.txt hardlink.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ ls -li original.txt hardlink.txt
2251799813686402 -rwxrwxrwx 2 ananya ananya 37 Sep 26 03:46 hardlink.txt
2251799813686402 -rwxrwxrwx 2 ananya ananya 37 Sep 26 03:46 original.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ echo "Data added through hard link." >> hardlink.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ cat original.txt
This is an inode demonstration file.
Data added through hard link.
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ ln -s original.txt symlink.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ ls -li original.txt hardlink.txt symlink.txt
2251799813686402 -rwxrwxrwx 2 ananya ananya 67 Sep 26 03:47 hardlink.txt
2251799813686402 -rwxrwxrwx 2 ananya ananya 67 Sep 26 03:47 original.txt
1970324836975747 lrwxrwxrwx 1 ananya ananya 12 Sep 26 03:47 symlink.txt -> original.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ stat symlink.txt
  File: 'symlink.txt' -> 'original.txt'
  size: 12          Blocks: 0          IO Block: 512    symbolic link
Device: 0,71      Inode: 1970324836975747 Links: 1
Access: (0777/lrwxrwxrwx)  Uid: ( 1000/  ananya)   Gid: ( 1000/  ananya)
Access: 2026-09-26 03:47:44.105224200 +0000
Modify: 2026-09-26 03:47:44.105224200 +0000
Change: 2026-09-26 03:47:44.106249500 +0000
 Birth: -
```

```

ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ stat -L symlink.txt
  File: symlink.txt
  size: 67          Blocks: 0          IO Block: 512   regular file
Device: 0,71    Inode: 2251799813686402  Links: 2
Access: (0777/-rwxrwxrwx)  Uid: ( 1000/  ananya)   Gid: ( 1000/  ananya)
Access: 2026-09-26 03:47:33.352462400 +0000
Modify: 2026-09-26 03:47:33.352462400 +0000
Change: 2026-09-26 03:47:33.352462400 +0000
 Birth: -
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ ls -li original.txt
2251799813686402 original.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ find . -inum 2251799813686402
./hardlink.txt
./original.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ stat original.txt
  File: original.txt
  size: 67          Blocks: 0          IO Block: 512   regular file
Device: 0,71    Inode: 2251799813686402  Links: 2
Access: (0777/-rwxrwxrwx)  Uid: ( 1000/  ananya)   Gid: ( 1000/  ananya)
Access: 2026-09-26 03:47:33.352462400 +0000
Modify: 2026-09-26 03:47:33.352462400 +0000
Change: 2026-09-26 03:47:33.352462400 +0000
 Birth: -
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ ls -li original.txt hardlink.txt symlink.txt
2251799813686402 -rwxrwxrwx 2 ananya ananya 67 Sep 26 03:47 hardlink.txt
2251799813686402 -rwxrwxrwx 2 ananya ananya 67 Sep 26 03:47 original.txt
1970324836975747 lrwxrwxrwx 1 ananya ananya 12 Sep 26 03:47 symlink.txt -> original.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ █

```

OBSERVATION:

The inode structure and link behavior were successfully investigated using `ls -li`, `stat`, and `find`. The original file initially had a unique inode number and a link count of 1. After creating the hard link, both `original.txt` and `hardlink.txt` were found to have the same inode number, and the link count increased to 2. Modifying the hard link also modified the original file, confirming that both names refer to the same inode and data. A symbolic link was then created, which had a different inode number and pointed to `original.txt`. The `stat -L` command followed the symbolic link and showed the inode of the original file. The `find -inum` command also successfully located both hard-linked filenames sharing the same inode.

Task- 2:

Design a program using mmap() to perform file reading and writing. Compare memory-mapped I/O with traditional read/write operations in terms of efficiency and implementation complexity.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>
#include <string.h>
#include <time.h>
#define BUFFER_SIZE 4096
/* Traditional read/write method */
void traditional_copy(const char *source, const char *destination)
{
    int src_fd, dest_fd;
    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;
    src_fd = open(source, O_RDONLY);
    if (src_fd < 0)
    {
        perror("Error opening source file");
        exit(1);
    }
    dest_fd = open(destination,
                    O_WRONLY | O_CREAT | O_TRUNC,
                    0644);
    if (dest_fd < 0)
    {
        perror("Error opening destination file");
        close(src_fd);
        exit(1);
    }

    while ((bytes_read = read(src_fd, buffer, BUFFER_SIZE)) > 0)
    {
        if (write(dest_fd, buffer, bytes_read) != bytes_read)
        {
            perror("Write error");
```

```

        close(src_fd);
        close(dest_fd);
        exit(1);
    }
}
close(src_fd);
close(dest_fd);
}
/* Memory-mapped I/O method */
void mmap_copy(const char *source, const char *destination)
{
    int src_fd, dest_fd;
    struct stat file_info;
    size_t file_size;
    char *src_map;
    char *dest_map;
    src_fd = open(source, O_RDONLY);
    if (src_fd < 0)
    {
        perror("Error opening source file");
        exit(1);
    }
    /* Get file size */
    if (fstat(src_fd, &file_info) < 0)
    {
        perror("fstat error");
        close(src_fd);
        exit(1);
    }
    file_size = file_info.st_size;
    dest_fd = open(destination,
        O_RDWR | O_CREAT | O_TRUNC,
        0644);

    if (dest_fd < 0)
    {
        perror("Error opening destination file");
        close(src_fd);
        exit(1);
    }
}

```

```

/* Set destination file size */
if (ftruncate(dest_fd, file_size) < 0)
{
    perror("ftruncate error");
    close(src_fd);
    close(dest_fd);
    exit(1);
}
/* Map source file into memory */
src_map = mmap(NULL,
               file_size,
               PROT_READ,
               MAP_PRIVATE,
               src_fd,
               0);
if (src_map == MAP_FAILED)
{
    perror("mmap source error");
    close(src_fd);
    close(dest_fd);
    exit(1);
}
/* Map destination file into memory */
dest_map = mmap(NULL,
                file_size,
                PROT_READ | PROT_WRITE,
                MAP_SHARED,
                dest_fd,
                0);
if (dest_map == MAP_FAILED)
{
    perror("mmap destination error");
    munmap(src_map, file_size);
    close(src_fd);
    close(dest_fd);
    exit(1);
}
/* Copy data directly through memory */
memcpy(dest_map, src_map, file_size);
/* Write mapped changes back to the file */

```

```

if (msync(dest_map, file_size, MS_SYNC) < 0)
{
    perror("msync error");
}
/* Unmap memory */
munmap(src_map, file_size);
munmap(dest_map, file_size);
close(src_fd);
close(dest_fd);
}
int main()
{
    const char *source = "original.txt";
    clock_t start, end;
    double traditional_time;
    double mmap_time;

    printf("=====\n");
    printf("Memory-Mapped I/O Performance Test\n");
    printf("=====\n\n");

    /*
     * Traditional read/write
     */
    printf("Traditional read/write started...\n");
    start = clock();
    traditional_copy(source, "traditional_copy.txt");

    end = clock();
    traditional_time =
        (double)(end - start) / CLOCKS_PER_SEC;
    printf("Traditional copy completed.\n");
    printf("Time taken: %.6f seconds\n\n",
        traditional_time)

    /*
     * Memory-mapped I/O
     */
    printf("Memory-mapped I/O started...\n");
    start = clock()

```

```

mmap_copy(source, "mmap_copy.txt");
end = clock();
mmap_time =
    (double)(end - start) / CLOCKS_PER_SEC;
printf("Memory-mapped copy completed.\n");
printf("Time taken: %.6f seconds\n\n",
    mmap_time);
/*
 * Comparison
 */
printf("=====\n");
printf(" Performance Comparison\n");
printf("=====\n");

printf("Traditional I/O : %.6f seconds\n",
    traditional_time);
printf("Memory-mapped I/O: %.6f seconds\n",
    mmap_time);
if (traditional_time < mmap_time)
{
    printf("\nTraditional I/O was faster in this test.\n");
}
else if (mmap_time < traditional_time)
{
    printf("\nMemory-mapped I/O was faster in this test.\n");
}
else
{
    printf("\nBoth methods took approximately the same time.\n");
}
return 0;
}

```

OUTPUT:

```

ananya@Ananya:~$ cd /mnt/d/OSSP_Programs/Practical/Week10
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ nano mmap_compare.c
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ gcc mmap_compare.c -o mmap_compare
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ ls
hardlink.txt  mmap_compare  mmap_compare.c  original.txt  symlink.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ ./mmap_compare
=====
Memory-Mapped I/O Performance Test
=====

Traditional read/write started...
Traditional copy completed.
Time taken: 0.000591 seconds

Memory-mapped I/O started...
Memory-mapped copy completed.
Time taken: 0.001850 seconds

=====
Performance Comparison
=====
Traditional I/O : 0.000591 seconds
Memory-mapped I/O: 0.001850 seconds

Traditional I/O was faster in this test.
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ ls -l original.txt traditional_copy.txt mmap_copy.txt
-rwxrwxrwx 1 ananya ananya 67 Sep 26 03:52 mmap_copy.txt
-rwxrwxrwx 2 ananya ananya 67 Sep 26 03:47 original.txt
-rwxrwxrwx 1 ananya ananya 67 Sep 26 03:52 traditional_copy.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ cat traditional_copy.txt
This is an inode demonstration file.
Data added through hard link.
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ cat mmap_copy.txt
This is an inode demonstration file.
Data added through hard link.
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ cmp original.txt traditional_copy.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ cmp original.txt mmap_copy.txt
ananya@Ananya:/mnt/d/OSSP_Programs/Practical/Week10$ █

```

OBSERVATION:

The program successfully implemented and compared traditional read()/write() I/O with memory-mapped I/O using mmap(). Both methods successfully copied the contents of original.txt into their respective output files, and the cmp commands produced no differences, confirming that the copied files matched the source. In this execution, traditional I/O took 0.000591 seconds, while memory-mapped I/O took 0.001850 seconds, so traditional I/O was faster for this particular small file. This shows that the performance of mmap() depends on factors such as file size and access pattern, and it is not necessarily faster for every workload.